Projeto Arquitetural: UberPB

1. Introdução e Objetivos

1.1. Visão Geral do Sistema

O "UberPB" é uma aplicação de simulação multimodal focada inicialmente no transporte de passageiros e motoristas, que foi expandida para englobar também um abrangente módulo de delivery de alimentos (simulando a dinâmica do Uber Eats). O sistema opera inteiramente através de uma interface de linha de comando (terminal), sem a necessidade de uma interface gráfica complexa. Originando-se do gerenciamento de entidades fundamentais como Passageiro, Motorista, Gerente, Veículo e Corrida, a aplicação agora também orquestra de forma robusta a interação e logística envolvendo Restaurantes, Entregadores, Cardápios, Pedidos (imediatos ou agendados) e um consolidado sistema de Avaliações mútuas.

1.2. Requisitos Arquiteturais (Drivers)

A arquitetura continuou sendo guiada pelos seguintes objetivos principais, que permitiram ao sistema escalar para receber o módulo de delivery de maneira coesa:

- **Estrutura e Escalabilidade:** O código é estruturado seguindo um padrão em camadas (*`Model-View-Service`*) para garantir que as responsabilidades (dados, apresentação e regras de negócio) sejam bem definidas. Essa separação foi crucial para permitir a adição dos novos serviços do módulo Uber Eats de forma orgânica e fácil de entender ou modificar.
- **Simplicidade de Implantação:** O sistema é auto suficiente, não exigindo a configuração e manutenção de um banco de dados externo. A persistência de dados (incluindo o armazenamento seguro de históricos de corrida, pedidos e menus de restaurantes) ocorre integralmente em arquivos locais.
- **Clareza no Tratamento de Erros:** O projeto utiliza exceções customizadas para mapear e lidar com os os fluxos de erro de negócio de forma explícita e controlada (fundamental para cenários como recusas de pedidos ou erros de categorias em veículos/entregadores).
- **Modularidade:** O uso estrito de interfaces, como o modelo `Repository` com sua implementação base (ex: `JsonRepository`), garante que a lógica de acesso a dados seja unificada. Isso abstraiu completamente os processos de manipulação de relatórios e arquivos JSON das regras do sistema em si.

---

2. Visão Lógica: Arquitetura em Camadas(Model-View-Service)

O sistema é estruturado em torno de um padrão de camadas que separa a representação da informação (Modelo) da interação do usuário com ela (Visão).

2.1. Detalhamento dos Componentes

- **View (Visão):**

- **Responsabilidade:** Apresentar os dados ao usuário e capturar suas interações. A View puramente exibe as informações e coleta os inputs, sem abrigar lógica de negócio em suas entranhas.
- **Implementação:** A View do sistema é inteiramente fundamentada na Interface de Linha de Comando (Terminal). A classe `Main.java` serve como ponto de entrada para toda a aplicação, inicializando a View (chamando `View.iniciar()`), onde a `MenuPrincipalView` detém o fluxo primário de desenho do menu principal. A responsabilidade por apresentar telas e capturar solicitações foi ramificada harmonicamente no pacote `org.example.view` para o transporte e o delivery, com classes dedicadas para cada tela (ex: `LoginView`, `SolicitarCorridaView`, `MenuPassageiroView`, além das novas `SolicitarPedidoView`, `MenuRestauranteView` e `MenuEntregadorView`).

- **Controle de Fluxo (na Camada View):**
  - **Responsabilidade:** Atuar como o intermediário direto entre a ação efetuada pelo usuário (Visão visualizada/interagida) e as diretrizes e regras sistêmicas (Modelo).
  - **Implementação:** O desenho e cadência da aplicação são gerenciados pelas instâncias de classe da própria `View`. Elas limpam e ingerem a entrada do usuário (utilizando objetos como `Scanner`, isolados em utilitários como `ViewUtils`), chamam métodos de negócio pertinentes da camada Service (ex: a `SolicitarCorridaView` chama o `CorridaService.solicitarCorrida`(...), assim como e a `MenuRestauranteView` delega as escolhas para o `PedidoService.confirmarPedido`(...)), para só então saltar para a próxima etapa na tela com base nessa resposta interna (ex: a `LoginView` encaminha para `MenuPassageiroView.exibir(passageiroLogado)` ou `MenuEntregadorView.exibir(entregadorLogado)`).

- **Model (Modelo):**
  - **Responsabilidade:** O coração do sistema. O Modelo é onde a mágica matemática e condicional inteira acontece, congregando a real estrutura de dados corporativa, as estritas regras de negócio, os métodos de validação das avaliações e toda a lógica de salvamento e recuperação (persitência).
  - **Implementação:** Essa camada é dividida rigidamente entre três subcomponentes basilares e co-dependentes:
    - **Entidades (`/model/entity`):** Moldam a malha central de informações que viajam pela aplicação. Consistem em classes maduras de herança, como `Usuario` (classe base) e suas derivações `Motorista`, `Passageiro` e agora os recém-criados `Entregador` e `Restaurante`. Englobam também entidades-satélite independentes como `Veiculo`, `Corrida`, `Pedido`, `Cardapio` e `Avaliacao`. Enums restritos, a exemplo do `StatusCorrida`, o novo `StatusPedido` e `CategoriaVeiculo`, solidificam em uma via de mão única todos os possíveis estados no ciclo de vida de um percurso ou lanche.
    - **Serviços (`/model/service`):** São formalmente o "cérebro" da malha de lógica de negócio e da verificação de requisitos. Arquivos como

`CorridaService`, `PassageiroService` e o recém implementado `PedidoService` (junto do `RestauranteService` e `EntregadorService`), consolidam em segurança todo o processo e garantem (por exemplo) o cálculo real do trajeto ou os critérios exigentes da tarifa dinâmica.

- **Repositórios (`/model/repository`):** Os guardiões do disco, cuidando unicamente de abstrair e resolver os dilemas práticos de persistência. A infraestrutura brilha em sua classe genérica `JsonRepository<T>`, que entende e salva serializações localmente em formato `.json` por intermédio da lib externa Gson. Implementações herdeiras mais diretas, como `MotoristaRepository`, relatórios do `CorridaRepository` ou o rastreio em `PedidoRepository` e `AvaliacaoRepository`, empoderam a linha de frente do serviço garantindo acesso veloz aos históricos.

---

3. Visão de Dados

3.1. Mecanismo de Persistência

Em prol da portabilidade e simplicidade, o sistema descarta dependências externas severas e utiliza arquivos de texto puro no formato estruturado `JSON` como seu "banco de dados" local (simulando instâncias literais no ambiente de execução do usuário). Cada tipo de entidade principal é rigorosamente isolado e armazenado de forma estanque em seu próprio arquivo dedicado (ex: `passageiros.json`, `motoristas.json`, `corridas.json` e, alicerçando o novo módulo do Uber Eats, os arquivos restaurantes.json, entregadores.json, pedidos.json e sua listagem unificada de `avaliacoes.json`). A aclamada biblioteca externa `Google Gson` atua de intermediária no processo, encarregando-se pela automação robusta da serialização e desserialização orgânica dos objetos orientados da liguagem Java, convertendo fluxos instanciados de classes em registros flexíveis de texto persistido em disco físico, e vice-versa.

4. Visão de Implantação (Deployment)

- **Ambiente de Execução:** Java Virtual Machine (JVM).
- **Empacotamento:** A aplicação é empacotada como um arquivo JAR executável.
- **Dependências Externas:**
  - **Runtime (JAR Principal):** Em prol de uma governança imutável, a única dependência de runtime da aplicação principal continua sendo a robusta biblioteca externa `com.google.code.gson`. É ela quem garante o controle minucioso e ininterrupto da persistência de dados estruturados do sistema em `JSON`, sustentando tanto o transporte original (motoristas/corridas) quanto a infraestrutura orgânica de restaurantes, pedidos de comida, avaliação e entregadores logados.
  - **Testes:** A suíte de testes de unidade utiliza o framework `org.junit.jupiter` (JUnit 5).

- **Visualização (Simulação):** A ramificação da funcionalidade de simulação explodiu para dois domínios complexos de tráfego. Originalmente provida pela abstração lógica (`SimuladorViagem.java`), que produz metadados e um arquivo processado ativamente pelo `Mapa.html`, a simulação passou a englobar também métricas da rota visual dinâmica no acompanhamento dos status do pedido. Assim, a roteirização provê a interface via terminal logada ou o arquivo mapa/HTML com projeções instantâneas ligando os dados entre entregador → restaurante → cliente.

5. Funcionalidades implementadas

5.1. Cadastro e autenticação

RF01: Permitir cadastro de passageiros e motoristas.O sistema deve permitir cadastro de passageiros, motoristas, restaurantes e entregadores.

Implementação: O cadastro é feito por meio de fluxos dedicados na camada de `View` (`CadastroPassageiroView.java`, `CadastroMotoristaView.java`). As regras de negócio (como validação de e-mail e CPF duplicados) são tratadas pelos respectivos serviços (`PassageiroService.java`, `MotoristaService.java`).

```
=== UberPB CLI - Menu Principal ===
1 - Cadastrar como Passageiro/Cliente
2 - Cadastrar como Motorista
3 - Cadastrar como Restaurante
4 - Cadastrar como Entregador
```

Figura 1

```
Escolha uma opção: 1
--- Cadastro de Passageiro ---
(Digite 'voltar' a qualquer momento para cancelar)
Nome completo: manoel
CPF (Formato: 000.000.000-00): 104.351.394-95
E-mail: manoel@gmail.com
Senha (mínimo de 6 caracteres): Senha123
Telefone (apenas números):
```

Figura 2

RF02: O sistema deve validar documentos e características do veículo para definir a categoria em que o motorista pode atuar (UberX, Comfort, Black, etc.) e entregadores.

Implementação: Durante o cadastro do motorista, o sistema exige um segundo passo (`CadastroVeiculoView.java`), onde os dados do veículo são coletados. O

`VeiculoService.java` contém a lógica `determinarCategoria` que, com base no ano, assentos e outros atributos, define a categoria do veículo (UberX, Comfort, etc.).

```
Olá, pedro! Agora, vamos cadastrar seu veículo.
Marca do veículo (ex: Chevrolet): Chevrolet
Modelo do veículo (ex: Onix): Onix
Placa (ex: ABC-1234 ou ABC1D23): ABC1D23
Renavam: 03007043419
Cor: Vermelho
Ano de fabricação: 2019
Número de assentos (ex: 5): 5
Capacidade do porta-malas em litros (ex: 280.5): 180
É um veículo premium/de luxo? (S/N): N

[INFO] Categoria do veículo determinada automaticamente: Uber Comfort

Veículo cadastrado com sucesso!
```

Figura 3

RF03: Permitir login com e-mail.

Implementação: A `LoginView.java` captura e-mail e senha e os repassa aos serviços `PassageiroService` e `MotoristaService` para autenticação.

```
Como você deseja logar?
1 - Como Passageiro
2 - Como Motorista
Escolha uma opção: 1

Email: manoel@gmail.com
Senha: Senha123

Login como Passageiro bem-sucedido! Pressione ENTER para continuar.

--- Menu do Passageiro ---
Olá, manoel! ⭐ Avaliação Média: 0,00
```

Figura 4

5.2. Solicitação de corrida

RF04, RF06: Inserir origem, destino e selecionar categoria.

Implementação: A `SolicitarCorridaView.java` gerencia este fluxo completo, permitindo ao passageiro selecionar uma origem e destino da lista (ou cadastrar um novo) e, em seguida, selecionar a categoria de veículo desejada..

```
0--- Solicitar Nova Corrida ---
0--- Escolha o local de origem ---
1 - Universidade Estadual da Paraíba (UEPB) - Campus I
2 - Universidade Federal de Campina Grande (UFCG)
3 - Parque do Povo
4 - Açude Velho (Monumento 'Os Pioneiros')
5 - Partage Shopping
6 - Aeroporto Presidente João Suassuna (SBKG)
7 - Terminal de Integração
8 - Vila Sítio São João
9 - Casa

10 - Adicionar Nova Localização
0 - Voltar/Cancelar

Escolha uma opção: |
```

Figura 5

```
0--- Escolha o local de destino ---
1 - Universidade Estadual da Paraíba (UEPB) - Campus I
2 - Universidade Federal de Campina Grande (UFCG)
3 - Parque do Povo
4 - Açude Velho (Monumento 'Os Pioneiros')
5 - Partage Shopping
6 - Aeroporto Presidente João Suassuna (SBKG)
7 - Terminal de Integração
8 - Vila Sítio São João
9 - Casa

10 - Adicionar Nova Localização
0 - Voltar/Cancelar

Escolha uma opção: 2
```

Figura 6

```
--- Selecione a Categoria do Veículo ---
1 - UberX        (Corrida mais econômica)
2 - Uber Comfort (Carros mais novos e espaçosos)
3 - Uber Black   (Veículos premium)
4 - Uber Bag     (Veículos com porta-malas maior)
5 - Uber XL      (Capacidade para mais passageiros)

Escolha uma opção (ou digite '0' para voltar): 2
```

Figura 7

RF05, RF07: Calcular estimativa de preço e notificar motoristas.

Implementação: O `CorridaService.java` centraliza esta lógica. O método `calcularPrecoEstimado` usa a fórmula (de `Localizacao.java`) e fatores de categoria/demanda. Após a confirmação, o método `solicitarCorrida` inicia o processo de busca, chamando `encontrarProximoMotoristaDisponivel` para notificar os motoristas elegíveis.

```
Calculando estimativa e verificando disponibilidade de motoristas...
Preço Estimado: R$ 10,76
```

Figura 8

5.3. Solicitação de Pedido (Uber Eats)

RF19: O passageiro (cliente) deve poder acessar restaurantes disponíveis próximos.

Implementação: A listagem e interação na interface são gerenciadas pela classe EscolherRestaurantesView.java. O controle de apresentação aciona o método `listarRestaurantes` da classe `RestauranteService.java` que por sua vez solicita todos os estabelecimentos registrados no banco através do repositório (`restauranteRepository.getAll()`). Após o usuário selecionar pelo ID o restaurante ativo na CLI de sua preferência, o sistema instancia uma ponte direta chamando o método `executar` da classe `RealizarPedidoView`, passando o restaurante selecionado e os dados do passageiro titular para a próxima etapa.

```
--- Escolha um restaurante ---

--------------------------------
Digite: 1
Nome: Pizza Boa
Categoria: Pizzaria
Endereço: null
--------------------------------

--------------------------------
Digite: 2
Nome: Pizzaria Central
Categoria: Pizzaria
Endereço: null
--------------------------------

--------------------------------
Digite: 3
Nome: Restaurante Teste
Categoria: Comida
Endereço: null
--------------------------------

--------------------------------
Digite: 4
Nome: Restaurante Teste
Categoria: Comida
Endereço: null
```

Figura 9

RF20: O cliente deve poder visualizar cardápios, preços, taxas de entrega e tempo estimado.

Implementação: Essa responsabilidade visual é centralizada na classe `RealizarPedidoView.java`. O cardápio e os preços individuais são resgatados por meio do `RestauranteService.java`, utilizando o método `getMenu()`, e imediatamente listados na tela. Após o cliente selecionar os pratos desejados e definir o destino, a mesma view invoca o `PedidoService.java`, chamando consecutivamente os métodos `calcularPrecoEstimado` e `calcularTempoDeEntrega`. Esses métodos efetuam os cálculos cruzando os dados da tarifa, dos itens e das coordenadas de localização do restaurante e cliente, retornando a estimativa exata de tempo e custo antes da tela prosseguir para a etapa de pagamento ou agendamento.

```
--- Cardapio de Pizza Boa ---
Id: 1
Nome: Pizza
Ingredientes: Massa
Preço: 40.0
Tempo de Preparo: 20

0 Itens adicionados ao Pedido.
```

Figura 10

RF21: O cliente deve poder montar o pedido e confirmar a compra.

Implementação: Essa lógica ocorre prioritariamente no fluxo interativo do `RealizarPedidoView.java`. O cliente digita sequencialmente os IDs dos pratos, populando a lista itensPedido (validada via `RestauranteService.getItemPorId()`). Finalizada a seleção, a view solicita o "ok" final e chama a interface `PagamentoPedidoView.executar()` para capturar a forma de pagamento. Obtendo o retorno positivo da transação, o método consolidador `realizarPedido` (localizado em `PedidoService.java`) é executado, recebendo o cliente, restaurante, pratos, valores e o destino para oficializar e salvar permanentemente a nova instância do `Pedido`

```
Escolha uma opção (Digite: 0, para concluir o pedido):  1

1 Itens adicionados ao Pedido.

Escolha uma opção (Digite: 0, para concluir o pedido):  1

2 Itens adicionados ao Pedido.

Escolha uma opção (Digite: 0, para concluir o pedido):  0
```

Figura 11

RF22: O sistema deve notificar o restaurante e o entregador mais próximo sobre o pedido.

Implementação: Essa automação ocorre no final do método principal `realizarPedido` através da classe `PedidoService.java`. Logo após injetar o registro inicial da solicitação, o sistema dispara a chamada para o método interno `encontrarProximoEntregadorDisponivel`. Utilizando filtros (`Streams`) e os validadores esféricos de `Localizacao.java`, é capturado o entregador de status livre mais perto do restaurante. Imediatamente a seguir, as regras atualizam as entidades em memória acionando `entregadorMaisProximo.adicionarPedidoNotificada(...)` e `restaurante.adicionarPedidoNotificado(...)`, finalizando a notificação instantânea quando ambos os objetos são sincronizados com o banco através de seus respectivos `Repositories`.

RF23: O sistema deve permitir pedidos agendados ou imediatos.

Implementação: Essa bifurcação de escolha é construída primordialmente na classe de exibição `RealizarPedidoView.java`. O sistema indaga na reta final do fluxo da interface se o cliente quer agendar a entrega. Quando afirmativo (com validações da hora inserida em formato texto para a formatação temporal `LocalDateTime`), a variável temporal é enviada preenchida ao invocar o núcleo do `PedidoService.java` (no método `realizarPedido`). Esse Service aciona uma estrutura condicional robusta de fábrica: quando há agendamento, ele instancia nativamente o

`Pedido` vinculando esse novo atributo de agendamento; senão, opta pelo preenchimento clássico setando a hora imediata via `LocalDateTime.now()`. Condicionais posteriores nesse serviço impedem a evolução de corridas agendadas antes da sua hora marcada.

```
Deseja agendar o horario de entrega do seu pedido? (S/N):
N
```

Figura 12

.

5.4. Aceite Corrida (ou Pedido)

RF08: Motorista poder aceitar ou recusar a corrida.

Implementação: O motorista, através da `CorridaView.java`, visualiza as corridas notificadas. Ele pode aceitar (chamando `CorridaService.aceitarCorrida`) ou recusar (chamando `MotoristaService.negarCorrida`).

```
--- Corridas Notificadas ---
ID: 13 | Origem: Universidade Estadual da Paraíba (UEPB) - Campus I | Destino: Universidade Federal de Campina Grande (UFCG) | Valor: R$ 10,76 (Dinheiro)

Digite o ID da corrida para interagir (ou 0 para voltar): 13
--- Detalhes da Corrida ID: 13 ---
Passageiro: manoel
Origem: Universidade Estadual da Paraíba (UEPB) - Campus I
Destino: Universidade Federal de Campina Grande (UFCG)
Categoria: Uber Comfort
Valor Estimado: R$ 10,76
Forma de Pagamento: Dinheiro

1 - Aceitar e Iniciar Corrida
2 - Negar Corrida
0 - Voltar
```

Figura 13

RF24: O entregador deve poder aceitar ou recusar pedidos de entrega.

Implementação: O entregador, através da classe `GerenciarPedidosEntregadorView.java`, visualiza os pedidos pendentes. Ele pode prosseguir aceitando a solicitação (chamando o método `PedidoService.aceitarPedido`) ou recusar a entrega (chamando `PedidoService.rejeitarPedidoEntregador`), o que direciona o pedido imediatamente para o próximo entregador livre na região.

```
=== Ver Pedidos Disponíveis ===
Entregador: Marcos Motoboy1
----------------------------------
ID Pedido: 11
Restaurante : Pizza Boa
Origem: null
Destino: Centro
Valor da Entrega: R$ 6173,16
----------------------------------
Digite o ID do pedido que deseja gerenciar (ou 0 para voltar):
11

Pedido #11 selecionado!
1 - Aceitar Pedido
2 - Rejeitar Pedido
0 - Voltar

Escolha uma opção: 1
```

Figura 14

RF25: O restaurante deve poder confirmar ou rejeitar pedidos recebidos.

Implementação: O restaurante, através da classe `GerenciarPedidosRestauranteView.java`, visualiza instantaneamente os novos pedidos recebidos. Ele pode confirmá-los para o preparo (chamando o método `RestauranteService.aceitarPedido`) ou rejeitá-los de imediato (chamando `RestauranteService.rejeitarPedido`), encerrando o fluxo e notificando o usuário.

```
Restaurante: Pizza Boa
----------------------------------
ID Pedido: 11
Cliente ID: 11
Valor: R$ 6173,16
Itens:
  - Pizza (R$ 40,00)
  - Pizza (R$ 40,00)
----------------------------------
ID Pedido: 12
Cliente ID: 11
Valor: R$ 6173,16
Itens:
  - Pizza (R$ 40,00)
  - Pizza (R$ 40,00)
----------------------------------
ID Pedido: 13
Cliente ID: 11
Valor: R$ 6173,16
Itens:
  - Pizza (R$ 40,00)
  - Pizza (R$ 40,00)
----------------------------------
Digite o ID do pedido que deseja gerenciar (ou 0 para voltar):
11

Pedido #11 selecionado!
1 - Aceitar Pedido (Iniciar Preparo)
2 - Rejeitar Pedido
0 - Voltar

Escolha uma opção: 1
```

Figura 15

RF09: Atribuir corrida ao motorista mais próximo/melhor avaliado.

Implementação: A lógica de atribuição está em `CorridaService.encontrarProximoMotoristaDisponivel`. O método filtra motoristas (disponíveis, na categoria correta, que não rejeitaram) e os ordena usando um comparador que prioriza avaliação (para categorias premium) ou distância (para UberX).

5.4. Acompanhamento da Corrida

RF10, RF11, RF12: Acompanhar localização e rota.

Implementação: O passageiro pode monitorar o status na AcompanharCorridasView.java. Quando a corrida é aceita, o SimuladorViagem.java é acionado, escrevendo coordenadas em dados_viagem.js. O Mapa.html (aberto pela MapaView.java) lê esse arquivo e exibe a simulação da rota (em linha reta) no navegador usando Leaflet. Estas funcionalidades foram concluídas nas Sprints 2 e 3.

```
0--- Menu do Motorista ---
Olá, pedro! ⭐ Avaliação Média: 0,00
Olá, pedro! Status: EM_CORRIDA
Localização Atual: Partage Shopping
0--- Gerenciando Corrida Atual ---
Passageiro: manoel
Status Atual: EM_CURSO

Abrindo mapa de acompanhamento no seu navegador...

--- Iniciando simulação da viagem no mapa ---
Por favor, verifique a janela do navegador que foi aberta.
Simulação: Passo 1 de 20... Posição: (-7,2049, -35,9113)
Simulação: Passo 2 de 20... Posição: (-7,2056, -35,9112)
Simulação: Passo 3 de 20... Posição: (-7,2064, -35,9110)
Simulação: Passo 4 de 20... Posição: (-7,2071, -35,9109)
Simulação: Passo 5 de 20... Posição: (-7,2079, -35,9107)
Simulação: Passo 6 de 20... Posição: (-7,2086, -35,9105)
Simulação: Passo 7 de 20... Posição: (-7,2094, -35,9104)
Simulação: Passo 8 de 20... Posição: (-7,2101, -35,9102)
```

Figura 16

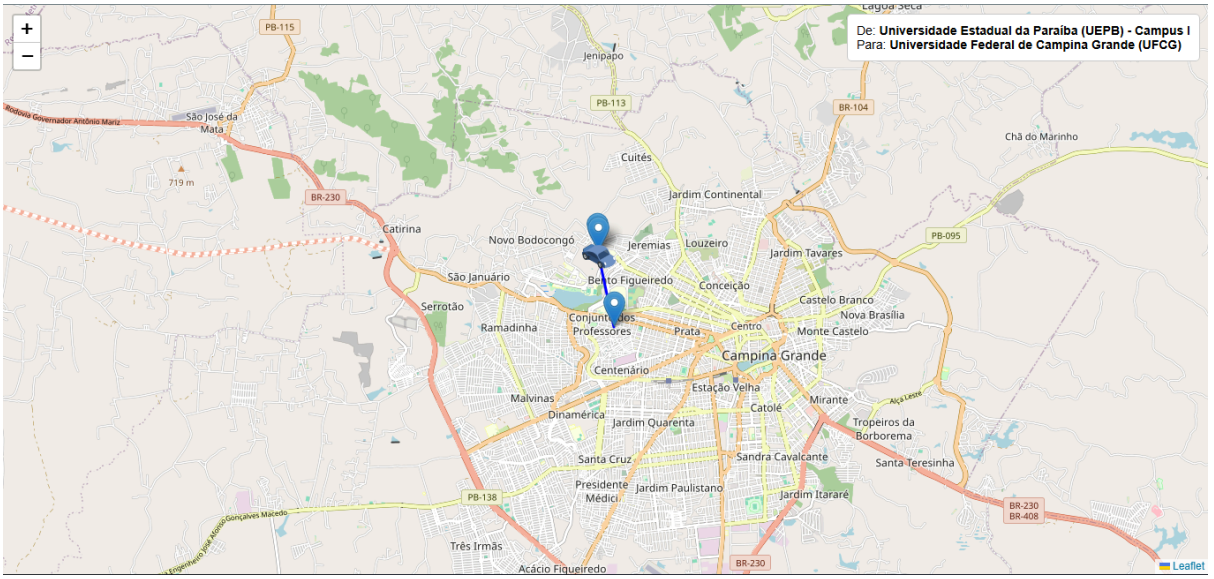

Figura 17

RF26: O cliente deve acompanhar o status do pedido (preparação, retirada, entrega, ...)

Implementação: O cliente, por meio da sua tela de acompanhamento na View logada, consome iterativamente o método `PedidoService.formatarStatusParaCliente()`. Esse método atua como tradutor amigável dos estágios internos estruturados no Enum (convertendo progressões precisas como `ACEITA`, `EM_PREPARO`, `SAIU_PARA_ENTREGA` e `EM_CURSO` para texto compreensível). Adicionalmente, esse avanço simulado opera de forma autônoma através da varredura cronológica feita por `PedidoService.avancarStatusAutomatico()`, que progride essas etapas de preparo e rota temporizado internamente pelo código.

RF27: O entregador deve visualizar a rota até o restaurante e até o cliente.

Implementação: Essa interatividade baseia-se na ferramenta de abstração `SimuladorViagem.java`. Logo após o condutor aceitar e gerenciar a entrega pela classe `GerenciarPedidosEntregadorView.java`, a engrenagem do simulador (`iniciarSimulacao`) entra em ação, injetando as coordenadas físicas (origem do restaurante e destino do cliente) e retroalimentando o painel dinâmico da rota e do fluxo veicular ao decorrer do tempo programado.

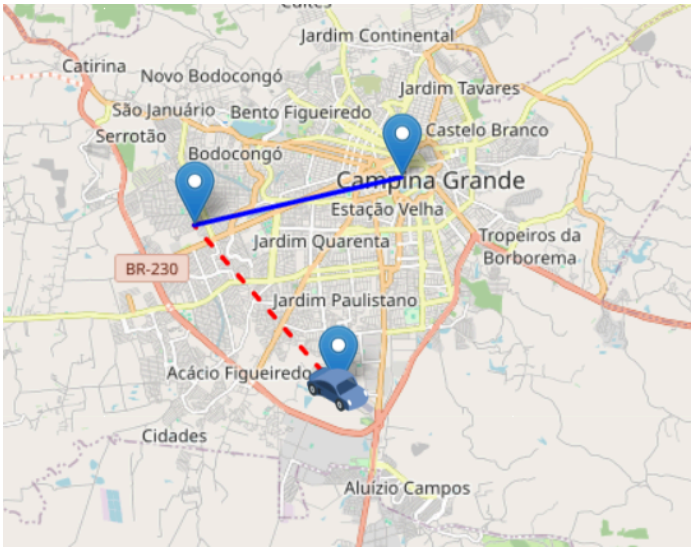

Figura 18

5.5. Pagamentos

RF13: Permitir pagamento via cartão, PIX, PayPal ou dinheiro.

Implementação: A `PagamentoView.java` gerencia a seleção do método de pagamento. A enum `FormaPagamento.java` confirma que todas as quatro formas (Cartão, PIX, PayPal, Dinheiro) estão implementadas.

RF14: Calcular automaticamente o valor da corrida.

Implementação: O `CorridaService.calcularPrecoEstimado` é usado para definir o valor da corrida no momento da solicitação.

```
Selecione a Forma de Pagamento:
1 - Cartão de Crédito
2 - PIX
3 - PayPal
4 - Dinheiro
0 - Cancelar Solicitação

Escolha uma opção:
```

Figura 19

RF15: Emitir recibo eletrônico ao final da corrida.

Implementação: O recibo é implementado como parte do histórico. O `HistoricoCorridasView.java` exibe os detalhes completos da viagem (origem, destino, valor, motorista, etc.) após sua conclusão.

```
--------------------------------
Data: 22/10/2025 às 20:54
Origem: Universidade Estadual da Paraíba (UEPB) - Campus I
Destino: Universidade Federal de Campina Grande (UFCG)
Valor Pago: R$ 10,76
Categoria: Uber Comfort
Forma de pagamento: Dinheiro
Motorista: pedro
--------------------------------
```

Figura 20

RF15: Emitir recibo eletrônico ao final da corrida.

Implementação: O recibo é implementado como parte do histórico. O `HistoricoCorridasView.java` exibe os detalhes completos da viagem (origem, destino, valor, motorista, etc.) após sua conclusão.

5.6. Avaliação

RF16: Permitir que passageiros e motoristas se avaliem mutuamente.

Implementação: Após a corrida, ambos os usuários são notificados. Os fluxos `AvaliacaoView.java` e `AvaliacaoPendentesView.java` gerenciam a coleta da nota (1-5). A nota é salva no objeto `Corrida` e a média do usuário é atualizada via `recalcularMediaAvaliacao` nos respectivos serviços.

```
0--- Avalie o Passageiro ---
Passageiro: manoel

De 1 a 5, qual a sua avaliação para o passageiro manoel? (5 é excelente): 5

Avaliação do passageiro salva! Pressione ENTER para voltar ao menu.
```

Figura 21

RF28: O sistema deve permitir que clientes, entregadores e restaurantes se avaliem mutuamente.

Implementação: Essa reciprocidade visual ocorre invariavelmente após ou independente das entregas concluídas, utilizando o terminal nas telas da camada de visão específicas para avaliações (ex: `AvaliacaoView.java`). Assim que os clientes, entregadores e restaurantes inserem os escopos de notas, o envio aciona diretamente os Services competentes de cada conta (exemplo: `RestauranteService.receberAvaliacao(...)`). A camada recalcula prontamente a média aritmética agregada daquele perfil e salva de volta na entidade persistida do respectivo repositório.

```
Escolha uma opção: 2
ID Pedido: 11
Restaurante : Pizza Boa
Cliente : john
Valor: R$ 6173,16
Itens:
  - Pizza (R$ 40,00)
  - Pizza (R$ 40,00)
-----------------------------------

Digite o ID do pedido para Avaliar (ou 0 para voltar): 11

Digite uma nota de 0 a 5 para o Cliente: 5

Digite uma nota de 0 a 5 para o Restaurante: 5

Avaliação enviada com sucesso!
Pressione ENTER para continuar...
```

Figura 22

RF17: Utilizar a média das avaliações para priorizar motoristas.

Implementação: Conforme descrito no RF09, o `CorridaService.java` já utiliza a `mediaAvaliacao` do motorista como um dos critérios de ordenação ao buscar o melhor motorista para a corrida.

5.7. Histórico

RF18: Manter histórico de corridas filtrável por categoria de carro.

Implementação: Corridas finalizadas são movidas para a lista `historicoCorridas` na entidade `Passageiro`. O `HistoricoCorridasView.java` exibe essa lista e implementa uma lógica de filtro que permite ao usuário visualizar apenas corridas de uma categoria específica (ex: "UberX").

```
--- Seu Histórico de Corridas ---

Deseja filtrar por categoria de veículo? (S/N): n

Exibindo todas as corridas.

---------------------------------
Data: 22/10/2025 às 20:54
Origem: Universidade Estadual da Paraíba (UEPB) - Campus I
Destino: Universidade Federal de Campina Grande (UFCG)
Valor Pago: R$ 10,76
Categoria: Uber Comfort
Forma de pagamento: Dinheiro
Motorista: pedro
---------------------------------
```

Figura 23

6. Relatório de teste

Sprint 1

Durante a Sprint 1, foram realizados os testes iniciais do sistema, com foco nas funcionalidades de **cadastro e autenticação de usuários**, além da **solicitação de corrida**.

No primeiro conjunto de testes, avaliou-se o **cadastro de novos usuários**, verificando se o sistema aceitava apenas dados válidos e se armazenava corretamente as informações. Foram testados diferentes cenários, incluindo campos em branco, senhas curtas e e-mails duplicados. No primeiro teste, o sistema apresentou uma falha, permitindo o cadastro com senha de apenas três caracteres. O erro foi corrigido com a adição de uma validação mínima de comprimento de senha. Após a correção, os testes foram reexecutados e aprovados.

Em seguida, testou-se o **módulo de autenticação**, confirmando que apenas usuários registrados conseguiam acessar o sistema. Inicialmente, ocorreu um problema no processo de comparação da senha criptografada, o que impedia o login de novos usuários. A falha foi

rastreada até a função de hash e devidamente ajustada. Após a correção, todos os testes de login e logout foram bem-sucedidos.

Por fim, foram realizados testes na **funcionalidade de solicitação de corrida**, simulando um passageiro solicitando uma viagem até determinado destino. O teste verificou se a corrida era criada com status inicial **SOLICITADA** e se os dados de origem, destino e categoria do veículo eram armazenados corretamente. Na primeira execução, o status da corrida não estava sendo atualizado no repositório. O problema foi corrigido e os testes seguintes confirmaram o funcionamento esperado.

Os resultados da Sprint 1 garantiram que o sistema possuía um fluxo básico de cadastro, login e solicitação de corrida funcionando corretamente, servindo de base sólida para as próximas etapas de desenvolvimento.

Sprint 2

Na Sprint 2, o foco foi validar o fluxo operacional da corrida, desde o **aceite por um motorista** até o **acompanhamento e finalização da viagem**.

Nos **testes de aceite**, simulou-se o cenário em que um motorista disponível aceita uma corrida. O sistema atualizou corretamente o status para **ACEITA** e vinculou o ID do motorista à corrida. Em seguida, foi testado o caso de **dois motoristas tentando aceitar a mesma corrida**. Inicialmente, o segundo motorista também conseguia aceitar, o que configurava um erro de concorrência. A falha foi corrigida com a atualização imediata do status no repositório após o primeiro aceite. Após o ajuste, o segundo motorista passou a receber uma resposta negativa, e o teste foi aprovado.

Já nos **testes de acompanhamento da corrida**, verificou-se o comportamento do sistema durante o início, andamento e término da viagem. Ao iniciar a corrida, o status mudou corretamente para **EM_CURSO**, e o horário de início foi registrado. No encerramento, o sistema atualizou o status para **FINALIZADA** e registrou o horário de término. O primeiro teste apresentou uma falha, pois o motorista não voltava ao status **DISPONÍVEL** após o término. O método finalizarCorrida() foi revisado e corrigido, garantindo o retorno automático à disponibilidade. Após a correção, o teste foi reexecutado e aprovado.

Esses testes também confirmaram que as informações de histórico do passageiro e do motorista eram atualizadas corretamente. Todos os casos foram finalizados com sucesso, garantindo um fluxo completo e estável entre solicitação, aceite e finalização de corrida.

Sprint 3

Na Sprint 3, foram implementadas e testadas funcionalidades relacionadas à **etapa financeira da corrida**, abrangendo o **cálculo do valor**, a **emissão do recibo eletrônico** e o **processo de pagamento**.

O primeiro teste verificou o **cálculo do valor da corrida**, considerando distância, tempo estimado e categoria do veículo. Foram realizadas simulações com diferentes distâncias e categorias (UberX, Comfort e XL). Em um primeiro momento, observou-se uma diferença no cálculo do valor para corridas curtas, devido a um arredondamento incorreto na conversão da distância. Após corrigir o método de cálculo, os resultados passaram a refletir os valores esperados e o teste foi aprovado.

Em seguida, foi testada a **emissão do recibo eletrônico**, gerado automaticamente após a finalização da corrida. O teste confirmou a criação do recibo com todos os dados da viagem — incluindo valores, horários, motorista e passageiro. No primeiro teste, o recibo estava sendo salvo com campos nulos em corridas recentes. A causa foi um atraso na gravação da corrida antes da emissão do recibo. O problema foi solucionado implementando uma verificação de sincronização, garantindo que o recibo só fosse gerado após o salvamento completo da corrida.

Por fim, foram realizados os testes da **funcionalidade de pagamento**, simulando tanto pagamentos bem-sucedidos quanto falhos. Inicialmente, uma corrida paga não estava sendo marcada corretamente como "paga" no repositório. Após ajustar a atualização do campo de status do pagamento, o teste foi repetido com sucesso.

Com isso, a Sprint 3 foi concluída com todas as funcionalidades financeiras testadas e funcionando corretamente, garantindo transparência, precisão e rastreabilidade no processo de pagamento e emissão de recibos.

Sprint 4

A Sprint 4 concentrou-se nas funcionalidades de **histórico de corridas** e **avaliações mútuas entre passageiros e motoristas**, voltadas à experiência do usuário e à confiabilidade do sistema.

Nos **testes do sistema de históricos**, foi verificado se todas as corridas concluídas eram registradas corretamente para passageiro e motorista. Foram simuladas múltiplas corridas seguidas e, após cada finalização, confirmou-se que os dados eram adicionados aos respectivos históricos. O primeiro teste apresentou uma falha, onde corridas canceladas também estavam sendo exibidas. O problema foi corrigido adicionando um filtro que inclui apenas corridas com status **FINALIZADA**. Após o ajuste, todos os testes passaram, garantindo a precisão das informações exibidas no histórico.

Em seguida, os **testes do sistema de avaliações** verificaram se passageiros e motoristas conseguiam avaliar-se mutuamente ao final da corrida, e se as notas eram corretamente salvas e utilizadas na média geral. Durante o primeiro teste, observou-se que as avaliações dos motoristas não estavam sendo persistidas no banco de dados. O erro foi rastreado até a função de atualização do perfil e corrigido. Após a correção, novas execuções confirmaram que as notas estavam sendo gravadas corretamente e refletidas nas médias individuais.

Com esses resultados, a Sprint 4 foi encerrada com êxito, garantindo o pleno funcionamento do histórico de corridas e do sistema de avaliações, consolidando a base de confiança e transparência entre usuários e motoristas.

Sprint 5

A Sprint 5 concentrou-se no desenvolvimento do módulo de delivery, com foco principal na funcionalidade de solicitação de pedidos e na implementação das operações de CRUD para as novas entidades do sistema, além da criação e execução de testes para garantir a qualidade das entregas.

Nos testes relacionados à funcionalidade de solicitação de pedidos, foi verificado se o fluxo completo, desde a preparação da CLI para recebimento de pedidos até a montagem e o agendamento, estava funcionando corretamente. Foram simuladas diferentes situações de criação de pedidos por novos usuários, validando a montagem dos itens com base no cardápio e o correto registro das informações. Inicialmente, foi identificado um problema no agendamento, onde pedidos não estavam sendo registrados com o horário correto. O erro foi corrigido ajustando a lógica de tratamento de datas, e após novas execuções, os pedidos passaram a ser armazenados corretamente com seus respectivos horários.

Em paralelo, foram realizados testes nas funcionalidades de CRUD de usuários e das novas entidades do sistema, como restaurantes e entregadores. Durante os testes de autenticação, foi identificado que novos usuários não conseguiam realizar login após o cadastro. O problema foi rastreado até a etapa de persistência das credenciais e corrigido. Após o ajuste, os testes confirmaram que o fluxo de criação e autenticação estava operando corretamente.

Por fim, também foram validadas as operações de criação das entidades de negócio, garantindo que os dados do cardápio e demais informações fossem corretamente cadastrados e recuperados pelo sistema. Com as correções aplicadas e todos os testes passando com sucesso, a sprint estabeleceu uma base sólida para a evolução do módulo de delivery.

Com esses resultados, a Sprint 5 foi encerrada com êxito, garantindo o pleno funcionamento do fluxo de solicitação de pedidos, das operações de CRUD das novas entidades e dos mecanismos de autenticação, consolidando uma base sólida e confiável para a evolução do sistema de delivery.

Sprint 6

A Sprint 6 concentrou-se na evolução das funcionalidades voltadas à experiência do usuário e ao fluxo completo de entrega, com destaque para a implementação do sistema de avaliações, acompanhamento de pedidos em tempo real e criação de painéis interativos para restaurantes e entregadores, além da ampliação da cobertura de testes. Nos testes unitários e de integração, foram validadas as funcionalidades do sistema de avaliações e o fluxo de aceite e recusa de pedidos. Durante os testes de integração, foram simulados diferentes cenários envolvendo múltiplos pedidos sendo aceitos e rejeitados por restaurantes e entregadores. Inicialmente, foi identificado um problema onde o status do pedido não era atualizado corretamente após a recusa, permanecendo disponível

indevidamente. O erro foi corrigido ajustando a lógica de transição de estados, e os testes subsequentes confirmaram o comportamento esperado. Também foram realizados aprimoramentos em testes anteriores, aumentando a confiabilidade geral do sistema. No desenvolvimento do sistema de avaliações, foram criadas as entidades e estruturas necessárias para armazenar as avaliações (ratings/reviews), além da implementação das interfaces em CLI que permitem ao cliente avaliar o restaurante e o entregador após a entrega, bem como possibilitam que entregadores e restaurantes também avaliem a experiência. Durante os testes iniciais, observou-se que algumas avaliações não estavam sendo persistidas corretamente. O problema foi corrigido na camada de repositório, garantindo o salvamento adequado dos dados.

Em relação ao acompanhamento e rota de entrega, foi implementada a lógica de visualização e simulação de rotas para o entregador, além da atualização dinâmica do status do pedido para o cliente. Nos primeiros testes, houve inconsistência na exibição do status em tempo real, que foi corrigida com ajustes na sincronização das atualizações, garantindo que o cliente visualize corretamente cada etapa do pedido.

Por fim, foram desenvolvidos painéis interativos em CLI para restaurantes e entregadores. O restaurante passou a visualizar pedidos pendentes e pode confirmá-los ou rejeitá-los, enquanto o entregador pode visualizar pedidos disponíveis e decidir aceitá-los ou recusá-los. Durante os testes, foi identificado que pedidos já aceitos ainda apareciam como disponíveis para outros entregadores. A falha foi corrigida com a implementação de um bloqueio adequado no momento da aceitação, garantindo a consistência do fluxo.

Com esses resultados, a Sprint 6 foi encerrada com êxito, garantindo o pleno funcionamento do sistema de avaliações, do acompanhamento de pedidos e dos fluxos de interação entre clientes, restaurantes e entregadores, consolidando uma experiência mais completa, confiável e interativa no sistema de delivery.